

Exercícios Desempenho

1. Para a seguinte declaração C, escreva o código assembly RISC-V correspondente. Suponha que as variáveis f , g e h já tenham sido colocadas nos registradores $x5$, $x6$ e $x7$, respectivamente. Use um número mínimo de instruções assembly RISC-V.

$f = g + (h - 5);$

2. Escreva uma única instrução C que corresponda às duas instruções de montagem RISC-V abaixo.

```
add f, g, h
add f, i, f
```

3. Para a seguinte declaração C, escreva o código assembly RISC-V correspondente. Suponha que as variáveis f , g , h , i e j sejam atribuídas aos registradores $x5$, $x6$, $x7$, $x28$ e $x29$, respectivamente. Suponha que o endereço base dos arrays A e B estejam nos registradores $x10$ e $x11$, respectivamente.

$B[8] = A[i-j];$

4. Para as instruções de montagem RISC-V abaixo, qual é a instrução C correspondente? Suponha que as variáveis f , g , h , i e j sejam atribuídas aos registradores $x5$, $x6$, $x7$, $x28$ e $x29$, respectivamente. Suponha que o endereço base dos arrays A e B estejam nos registradores $x10$ e $x11$, respectivamente.

```
slli x30, x5, 3          // x30 = f*8
add x30, x10, x30        // x30 = &A[f]
slli x31, x6, 3          // x31 = g*8
add x31, x11, x31        // x31 = &B[g]
lw x5, 0(x30)            // f = A[f]

addi x12, x30, 8
lw x30, 0(x12)
add x30, x30, x5
lw x30, 0(x31)
```

5. Mostre como o valor `0xabcdef12` seria organizado na memória de uma máquina *little-endian* e uma *big-endian*. Suponha que os dados sejam armazenados começando no endereço 0 e que o tamanho da palavra seja 4 bytes.

6. Suponha que os registradores x5 e x6 contêm os valores 0x80000000 e 0xD0000000, respectivamente.

- a. Qual é o valor de x30 para o seguinte código de montagem:

```
add x30, x5, x6
```

- b. O resultado em x30 está correto ou houve estouro?

- c. Para o conteúdo dos registradores x5 e x6, conforme especificado acima, qual é o valor de x30 para o seguinte código assembly?

```
sub x30, x5, x6
```

- d. O resultado em x30 está correto ou houve estouro?

- e. Para o conteúdo dos registradores x5 e x6 conforme especificado acima, qual é o valor de x30 para o seguinte código assembly?

```
add x30, x5, x6  
add x30, x30, x5
```

- f. O resultado em x30 está correto ou houve estouro?

7. Suponha o seguinte conteúdo de registro:

```
x5 = 0x0000AAAA, x6 = 0x12345678
```

- a. Para os valores de registro mostrados acima, qual é o valor de x7 para a seguinte sequência de instruções?

```
slli x7, x6, 4
```

- b. Para os valores de registro mostrados acima, qual é o valor de x7 para a seguinte sequência de instruções?

```
slli x7, x5, 4  
or x7, x7, x6
```

- c. Para os valores de registro mostrados acima, qual é o valor de x7 para a seguinte sequência de instruções?

```
srli x7, x5, 3  
andi x7, x7, 0xEF
```

8. Para a seguinte declaração C, escreva uma sequência mínima de instruções de montagem RISC-V que execute a operação idêntica. Suponha que x6 = A, e x17 é o endereço base de C.

```
A = C[0] << 4;
```

9. Suponha que o contador do programa (PC) esteja definido como 0x20000000.
- Qual intervalo de endereços pode ser alcançado usando a instrução `jump-and-link (jal)` do RISC-V? (Em outras palavras, qual é o conjunto de valores possíveis para o PC após a execução da instrução de salto?)
 - Qual intervalo de endereços pode ser alcançado usando a instrução RISC-V `branch if equal (beq)`? (Em outras palavras, qual é o conjunto de valores possíveis para o PC após a execução da instrução `branch`?)

10. Considere uma nova instrução proposta chamada **rpt**. Esta instrução combina a verificação de condição e o decremento do contador de um loop em uma única instrução. Por exemplo, **rpt x29, loop** faria o seguinte:

```
if (x29 > 0) {  
    x29 = x29 - 1;  
    goto loop  
}
```

- Se esta instrução fosse adicionada ao conjunto de instruções RISC-V, qual seria o formato de instrução mais apropriado?
- Qual é a sequência mais curta de instruções RISC-V que executa a mesma operação?

```
if (x29 > 0) {  
    x29 = x29 - 1;  
    goto loop
```

11. Traduza o seguinte código C para código assembly RISC-V. Use um número mínimo de instruções. Suponha que os valores de **a**, **b**, **i** e **j** estejam nos registradores **x5**, **x6**, **x7** e **x29**, respectivamente. Além disso, suponha que o registrador **x10** contenha o endereço base do array **D**.

```
for(i=0; i<a; i++)  
    for(j=0; j<b; j++)  
        D[4*j] = i + j;
```

12. Traduza o seguinte loop para C. Suponha que a variável **i** seja mantida no registrador **x5**, **x6** contém a variável **resultado** e **x10** contém o endereço base do array **MemArray**.

```
addi x6, x0, 0  
addi x29, x0, 100  
LOOP:  
    lw x7, 0(x10)  
    add x5, x5, x7  
    addi x10, x10, 4  
    addi x6, x6, 1  
    blt x6, x29, LOOP
```

13. Reescreva o loop do Exercício anterior para reduzir o número de instruções RISC-V executadas. Dica: Observe que a variável `i` é usada somente para controle de loop.
14. Implemente o seguinte código C em assembly RISC-V. Dica: Lembre-se de que o ponteiro da pilha deve permanecer alinhado em um múltiplo de 8.

```
int fib(int n){
    if (n==0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}
```